04 Spotify ETL Project Automated Data Pipeline

Spotify ETL Project: Building an Automated Data Pipeline

This project outlines the steps to build an automated Extract, Transform, Load (ETL) data pipeline for Spotify data using AWS services.

1. Data Movement: From "`to_processed`" to "`process`" Folder

After raw data is put into a "`to_processed`" folder, the next step involves moving it to a "process" folder. This is done by **first copying the data and then deleting the original data**.

- **Tools**:
  - A `boto3 resource object` is created for working at the resource level in AWS, similar to how a client object is used.
- **Process**:
  1. The code iterates through a list of `Spotify key` (file names) that were stored when the data was initially read.
  2. A dictionary is created containing bucket information and the individual key being processed.
  3. The function `s3_resource.meta.client.copy_from` is used to copy data.
     - **Source Bucket**: The current S3 bucket.
     - **Target Bucket**: The same S3 bucket, though it's possible to copy to a different bucket.
     - **Target Path**: The "process" folder path, attaching the extracted file name (e.g., `process/your_file_name.json`). The original file name is extracted using `key.split by slash`.
  4. After copying, the object is **deleted** from the "two process" folder using `bucket.delete_object` and providing the entire key.
  - **Important Note**: This copy and delete operation **must be placed outside the loop** where data is written. It ensures the operation happens once after all data for that run is processed.
  - **Result**: If there are no files, the "two process" folder will be deleted; otherwise, it will be emptied. The processed data will then be present in the "process" folder.

2. Automating the Pipeline with AWS Lambda Triggers

The entire ETL pipeline can be automated by setting up triggers for the Lambda functions.

a. Spotify API Extract Lambda Function Trigger

This function extracts data from the Spotify API and stores it in the "two process" folder.

- **Trigger Type**: **CloudWatch Events**.
- **Rule Creation**: Create a new rule.
- **Schedule**:
  - For testing: `rate(1 minute)`. This ensures the function runs every minute, and new data appears in the "`to_processed`" folder.

  - In a real-world scenario: This should be adjusted to `once a day, four times a day, or hourly`, as Spotify playlists don't update every minute.
  - Can also use **Cron expressions** for specific times, which take six fields: minutes, hours, day of month, month, day of week.
- **Purpose**: This trigger ensures that the **extract function runs periodically**, putting new raw data into the "`to_processed`" folder.
- **Cost Management**: It is highly recommended to **delete these triggers after testing** to avoid incurring unnecessary AWS charges, especially for frequent triggers like every 1 minute.

b. Spotify Transformation Lambda Function Trigger

This function transforms the raw data and moves it to the "process" folder.

- **Trigger Type**: **S3 Trigger**.
- **Configuration**:
  - **Bucket Name**: Specify the S3 bucket (e.g., `spotify_etl ...` is mentioned as the bucket name).
  - **Event Type**: `All object created` (or `ObjectCreated`). This means the Lambda function will run whenever a new object is created in the specified S3 location. Other event types like "object deleted" are also available.
  - **Prefix**: Set to the "`to_processed`" folder (`to_processed`) to specify the location where new objects will trigger the function.
  - **Suffix**: Set to `.JSON` to only trigger for JSON files.
- **Permissions**: It is **crucial to change the Lambda function's permissions** to allow communication between the extract and transformation Lambda functions.
  - Go to Configuration -> Permissions -> Lambda role.
  - Click on the Lambda role and then "Add permission" or "Attach policies" to grant the necessary access to the Lambda role.
- **Pipeline Flow**:
  1. The **extract function runs every 1 minute** (due to CloudWatch trigger) and deposits raw data into the "two process" folder.
  2. Whenever a new file is created in "two process" (and is JSON), the **S3 trigger activates the transformation function**.
  3. The transformation function processes the data, saves the transformed data into different designated folders, and **moves the original raw data from `two process` to `process`** to prevent re-processing.
- **Result**: This creates a continuous pipeline where new data is extracted, transformed, and moved, constantly updating the processed data.

3. Cleaning S3 Bucket for Fresh Data

Before building the Glue crawler, it is highly recommended to **clean the S3 bucket** to ensure a fresh start with data.

- **Folders to Clean**:
  - `album data`
  - `artist data`
  - `songs data`

  - `raw data` (specifically processed files)
- **Action:** Select the data within these folders and choose "**permanently delete**".
- **Reason:** This ensures that when AWS Glue is configured, it processes a clean set of data from scratch, avoiding issues with previously processed or erroneous files.

4. Modifying Lambda Function for Glue Crawler Compatibility

To ensure the Glue crawler can properly detect the schema, a small but important change is needed in the transformation Lambda function.

- **Change:** Add `index=False` to the `.to_csv()` method calls for:
  - `song_DF.to_csv()`
  - `album_DF.to_csv()`
  - `artist_DF.to_csv()`
- **Reason:** If the CSV files contain an index, the **Glue crawler will not be able to detect the entire schema correctly.** Removing the index ensures proper schema detection by Glue.

5. Building the Glue Crawler (Load Phase)

The Glue crawler is crucial for the "Load" phase of the ETL pipeline.

- **Purpose of Glue Crawler:**
  - Reads files stored in S3 buckets.
  - **Extracts schema information** (column names, data types) from these files.
  - **Creates tables in a data catalog**, which can then be used by Amazon Athena for querying.
- **Steps to Create a Crawler:**
  1. Go to the AWS Glue service and select "**Crawlers**" on the left side.
  2. Click "**Create crawler**".
  3. **Name the crawler** (e.g., `Spotify songs crawler`).
  4. **Add Data Source:**
     - Browse S3 and select the **specific folder** containing the transformed data (e.g., `transformed data/song/`).
     - **Crucially, add a slash (/) at the end of the folder path** to include all files within that folder.
  5. **Choose an IAM Role:**
     - You can **create a new IAM role** (e.g., `AWS Spotify S3 glue role`).
     - **Best Practice:** For least privilege, it's recommended to **create a new IAM role for each crawler** if the access is folder-level, as previous roles might only have permissions for specific folders. Alternatively, you can modify an existing role's permissions.
     - Ensure the role has permissions to access S3.
  6. **Add Database:** Create a new database (e.g., `Spotify DB`) where the tables will be stored in the Glue Data Catalog.
  7. **Run the Crawler:** After creation, run the crawler. You can monitor its status (e.g., `stopping stage`).
  - **Repetition:** This process needs to be **repeated for each data type** (e.g., songs, albums, artists) to create separate tables for each.

6. Verifying Crawled Data with Amazon Athena

Once the Glue crawler finishes, the data catalog is updated, and data can be queried using Athena.

- **Verification Steps:**
  1. In AWS Glue, go to "**Tables**" to see the newly created tables (e.g., `songs data`). Click on the table to view the crawled columns (e.g., Song ID, song name, duration, URL) and the location of the files.
  2. Open **Amazon Athena**.
  3. Select the **Spotify database** (`Spotify DB`).
  4. Click on the table name (e.g., `songs`) and choose "**Preview table**".
  5. You should now see all your data, enabling you to **query it using SQL**. This demonstrates reading S3 data using SQL queries.

7. Troubleshooting Glue Crawler Issues

Sometimes, Glue crawlers may not detect schema or headers correctly, requiring manual adjustments.

- **Incorrect Column Name Detection:**
  - **Problem:** Glue might not detect column names properly (e.g., for the `artist` table, it might not identify `artist ID`, `artist name`, `external URL`).
  - **Solution:** **Manually update the column names** in the Glue table definition.
- **Skipping Header Row:**
  - **Problem:** If your CSV files have a header row, Glue might treat it as data, including it in the table.
  - **Solution:** Go to the **"Advanced properties"** of the table in Glue, click "Action" -> "Edit table", and **add a new property**:
    - **Key:** `skip.header.line.count`
    - **Value:** `1`
  - **Reason:** This property explicitly tells Glue to **skip the first row** of the file, recognizing it as a header.
- **Verification:** After making these changes, preview the table in Athena again to ensure the data and column names are correct and the header is skipped. An example SQL query is shown to demonstrate querying the album data.

8. End-to-End Pipeline Summary

The completed pipeline automates the data flow from extraction to querying, providing a continuous data update mechanism.

1. **Data Extraction:** Python code extracts data from the Spotify API.
2. **Lambda Trigger (Extract):** A **CloudWatch trigger** runs the extract Lambda function **every 1 minute** (for testing purposes), storing raw data in the `two process` folder on Amazon S3.
3. **S3 Trigger (Transform):** Whenever new data arrives in the `two process` folder, an **S3 trigger activates the transformation Lambda function**.
4. **Data Transformation:** The transformation function processes the data, saves the transformed data into **different designated folders** (e.g., for songs, albums, artists), and **moves the original raw data from `two process` to `process`** to prevent re-processing.
5. **Glue Crawler:** The Glue crawler automatically **detects when new data arrives** in the transformed folders and **updates the data catalog** in AWS Glue.

6. **Amazon Athena**: Users can then **query the updated data** in the catalog using Amazon Athena with SQL queries. The count of rows in Athena increases as new data is added by the automated pipeline, demonstrating its continuous nature.

This project demonstrates an **advanced level of data engineering using Python** and provides a real-world example of how such a pipeline operates. While other tools like Airflow or Spark functions exist for data pipelines, this project focuses on a Python-centric approach.